

프로젝트 명세서: 가상 조립라인 품질관리 시뮬레이터 (v3)

0. 이 문서의 사용법 (Claude Code에게)

이 명세서는 단계별(Phase) 구현을 전제로 작성되었다. 반드시 **Phase 순서대로 구현하고, 각 Phase의 완료 기준을 실행으로 검증한 뒤 다음 Phase로 진행할 것**. 한 번에 전체를 구현하지 말 것. 각 Phase 완료 시 무엇을 어떻게 확인했는지 보고할 것.

1. 프로젝트 개요

웹 브라우저에서 가상의 2공정 직렬 조립라인이 가동되는 시뮬레이터를 만든다. 사용자가 생산 박스 수(1박스 = 반제품 10개)와 **생산 조건(온도, 헤드 속도, 레일 속도)**을 설정하고 시작하면, 반제품들이 인덱스(간헐 이송) 컨베이어를 따라 흐르며 두 대의 자동 나사 체결기가 완전히 동기화된 상하 운동으로 스크류를 체결한다. 체결 순간마다 측정값이 생성되어 실제 엑셀(.xlsx) 파일에 기록되며, **생산 조건이 측정값 분포에 영향을 주어 불량률이 달라진다**. 규격을 벗어난 측정값은 빨간 셀로 표시되고 해당 반제품 ID가 별도 추적된다. Lot 완료 시 조건이 명시된 품질 보고서가 자동 생성되고(사내 LLM 해석 코멘트 포함), **완료된 Lot 두 개를 선택해 조건별 품질을 비교하는 비교 보고서**를 만들 수 있다.

교육적 배경: MES의 핵심 개념인 데이터 수집, 품질 추적성(traceability), Lot 관리, 이상 감지에 더해, 공정 조건이 품질에 미치는 영향을 실험적으로 비교하는 공정 최적화 개념을 미니어처로 구현한다.

2. 기술 스택 (고정)

- 백엔드:** Python 3.11+, FastAPI, uvicorn
- 엑셀 기록:** openpyxl (실제 .xlsx 파일 생성 필수)
- 프론트엔드:** 순수 HTML + CSS + JavaScript (프레임워크 없음), 애니메이션은 SVG + JS
 - FastAPI가 정적 파일로 서빙 (별도 프론트 빌드 도구 금지)
- LLM 연동:** OpenAI 호환 API (사내 gpt-oss 서버), **openai** 파이썬 패키지 사용
- 실시간 통신:** 프론트 → 백엔드는 REST(fetch). 생산 현황 갱신은 폴링(0.5초 간격)으로 단순하게 구현 (WebSocket 금지 — 복잡도 최소화)

3. 도메인 용어 및 모델

용어	정의
박스(Box)	생산 입력 단위. 1박스 = 반제품 10개. 입력 범위 1~5박스 (총 10~50개)
Lot	한 번의 생산 실행 단위. 시작 시 LOT-YYYYMMDD-NNN 형식 ID 부여
생산 조건	Lot 시작 전에 설정하는 환경 변수 3종: 온도(°C), 헤드 속도, 레일 속도. Lot당 고정 — 생산 중 변경 불가

용어	정의
반제품(Unit)	라인을 흐르는 개별 제품(식기세척기 도어). SN-0001 부터 순번 부여
공정 (Station)	직렬 배치된 작업 지점 2개. ST1(기계 M1), ST2(기계 M2)
슬롯(Slot)	인덱스 컨베이어의 정지 위치. IN → ST1 → MID → ST2 → OUT 5개 (OUT = 대차 위치)
택트(Tact)	라인 전체가 공유하는 한 번의 작업 사이클 (하강→체결→상승→이송)
측정값	스크류 체결 토크(N·m). 공정마다 별도 측정 — 한 반제품당 측정 2건
판정	측정값이 규격 내면 OK, 벗어나면 NG. ST1, ST2 중 하나라도 NG면 최종 NG
USL / LSL	규격 상한 / 하한

반제품 처리 규칙 (중요): NG 반제품도 라인에서 배출되지 않고 끝까지 흘러간다. 화면상 색 변화 등 어떤 시각적 구분도 없으며, NG 여부는 데이터(엑셀, 현황 패널)에만 기록된다.

4. 설정값 (config.py로 분리, 모두 수정 가능해야 함)

python

```
# 생산 단위
BOX_SIZE = 10                # 1박스당 반제품 수
MAX_BOXES = 5                # 최대 박스 수

# 공정별 측정 스펙 (기준 분포)
ST1_SPEC = {"name": "체결토크1", "mean": 12.0, "std": 0.55, "lsl": 11.0, "usl": 13.0}
ST2_SPEC = {"name": "체결토크2", "mean": 8.0, "std": 0.45, "lsl": 7.2, "usl": 8.8}
# 위 std는 데모용(적정 조건에서 공정당 NG율 약 7%). 실공정 수준은 std 0.35 / 0.25

# — 생산 조건: 온도 —
TEMP_MIN, TEMP_MAX = 20, 30    # 설정 가능 범위 (°C, 1도 단위)
TEMP_OK_RANGE = (22, 26)      # 적정 온도 범위 (이 안이면 영향 없음)
TEMP_BIAS = 0.5                # 적정 범위 초과 시 측정값 +0.5, 미달 시 -0.5 (치우침)
TEMP_DEFAULT = 24

# — 생산 조건: 속도 (3단계 프리셋) —
# 헤드(체결) 속도: 택트의 하강/체결/상승 시간 배율 + 측정값 표준편차 배율(산포)
HEAD_SPEED_PRESETS = {
    "느림": {"time_factor": 1.3, "std_factor": 0.9},
    "보통": {"time_factor": 1.0, "std_factor": 1.0},
    "빠름": {"time_factor": 0.6, "std_factor": 1.5},
}
# 레일 속도: 이송(INDEX_SEC) 시간 배율만. 품질(측정값)에는 영향 없음
CONVEYOR_SPEED_PRESETS = {"느림": 1.3, "보통": 1.0, "빠름": 0.6}
SPEED_DEFAULT = "보통"

# 택트 타이밍 기준값 (초, 위 배율이 곱해짐)
HEAD_DOWN_SEC = 0.4
FASTEN_SEC = 0.6
HEAD_UP_SEC = 0.4
INDEX_SEC = 0.8

# 파일 경로
OUTPUT_DIR = "./output"        # Lot별 엑셀·보고서·비교보고서 저장 위치
LOTS_INDEX = "./output/lots_index.json" # 완료 Lot의 조건+집계 영구 저장 (비교 기능)

# LLM (환경변수에서 읽기, .env 파일 지원)
LLM_BASE_URL = env("LLM_BASE_URL") # 예: http://내부서버:8000/v1
LLM_MODEL = env("LLM_MODEL")        # 예: gpt-oss-20b
LLM_API_KEY = env("LLM_API_KEY", default="none")
```

5. 기능 요구사항

F1. 생산 조건 설정 및 시작

- 화면 상단에 **생산 조건 설정 패널**:
 - 박스 수: 1~5 스텝퍼, 옆에 "1박스 = 10개 · 총 N개 생산" 실시간 환산 표기
 - 온도: 20~30°C, 1도 단위 스텝퍼 또는 슬라이더. 적정 범위(22~26)를 UI에 표시(눈금 색 구분 등)하여 사용자가 실험 조건을 인지할 수 있게
 - 헤드 속도: [느림 | 보통 | 빠름] 3단계 버튼
 - 레일 속도: [느림 | 보통 | 빠름] 3단계 버튼
 - [생산 시작] 버튼
- 시작 시 `(POST /api/lot)` body: `{boxes, temperature, head_speed, conveyor_speed}` → Lot ID 발급, 엑셀 파일 생성
- **생산 중에는 조건 패널 전체 비활성화(잠금). 조건은 Lot당 고정이다**
- 참고: 박스는 입력 단위일 뿐이며 화면에 박스 그림을 그리지는 않는다 (반제품은 투입구 구조물에서 나온다 — F2 참조)

F2. 2D 조립라인 애니메이션 (SVG)

레이아웃 (좌→우): 투입구(IN) → 기계 M1 → 중간 구간(MID) → 기계 M2 → 대차(OUT). 하단에 컨베이어 레일이 전체를 관통한다.

기계 묘사 (M1, M2 완전 동일한 형태):

- 실제 자동 나사 체결기(탁상형 스크류 로킹 머신)를 아이콘 수준으로 단순화한 2D 측면 뷰
- 구성 3요소만: ① 바닥 베이스 프레임(납작하고 넓은 직사각형, 컨베이어를 감싸는 받침) ② 뒤쪽 수직 칼럼(세로로 긴 직사각형 기둥) ③ 칼럼 앞면을 따라 상하 슬라이드하는 체결 헤드
- 체결 헤드: 헤드 몸체(작은 사각 블록) 아래로 가늘고 뾰족한 드라이버 비트가 돌출된 형태. 하강 시 비트 끝이 반제품 윗면에 닿는 높이까지 내려옴
- 스크류 피더, 제어박스, 배선 등 그 외 요소는 그리지 않음
- 색상: 프레임·칼럼은 산업설비 느낌의 밝은 청회색 계열, 헤드는 구분되는 진한 색. 모서리는 약간 둥글게 하여 아이콘 느낌 유지

반제품(식기세척기 도어) 묘사:

- 가로로 길다란 막대 2개가 위아래로 겹쳐 쌓인 형태. 위 막대는 아래 막대보다 좌우 폭이 약간 짧고(약 85%) 중앙 정렬되어 단차가 보이게
- 스테인리스 금속 질감: SVG linearGradient로 밝은 은백색 → 중간 회색 → 밝은 은백색의 세로 방향 광택을 두 막대 모두에 적용
- **체결 완료 표시:** ST1 체결 완료 시 위 막대의 왼쪽 지점에, ST2 체결 완료 시 오른쪽 지점에 작은 나사 머리(진한 회색 원 + 십자 홈 선)가 나타난다. 이 표시는 대차에 들어가는 순간까지 유지된다
- 번쩍임, 파티클 등 다른 이펙트는 일절 넣지 말 것

투입구(IN) 묘사:

- 라인 왼쪽 끝에 투입구 구조물을 그린다: 개구부(어두운 사각 입구)가 뚫린 간단한 슈트/게이트 형태로, 컨베이어 시작점을 덮는 모습
- 새 반제품은 이송 단계에서 이 개구부로부터 미끄러져 나와 IN 슬롯에 자리 잡는 짧은 등장 애니메이션으로 나타난다. 허공에서 갑자기 생겨나는 연출 금지

완성 구역(OUT) — 대차 묘사:

- 라인 오른쪽 끝에 대차를 그린다: 간단한 2D 측면 뷰의 운반 카트(윗면이 열린 몸체 + 바퀴 2개 + 손잡이)
- ST2를 통과한 반제품은 이송 단계에서 대차 안으로 미끄러져 들어가며 화면에서 사라진다. **대차 안에 쌓이는 연출은 하지 않는다**
- 대신 대차 옆 또는 위에 완성 수량 카운터를 크고 명확한 숫자로 상시 표시한다: "완성 n / N" 형식. 진행 상황은 이 숫자와 현황 패널(F5)로만 파악한다

택트 동기화 규칙 — 이 애니메이션의 핵심 요구사항: 두 기계의 헤드는 반제품 유무와 관계없이 **항상 완전히 동일한 타이밍으로 움직인다** (빈 슬롯 위에서도 똑같이 공타). 라인 전체가 아래 4단계 상태 머신 하나를 무한 반복한다. 각 단계의 소요 시간은 기준값 × 해당 Lot의 속도 배율(헤드 속도 → 1~3단계, 레일 속도 → 4단계):

- 하강:** 두 헤드가 동시에 내려간다 ($HEAD_DOWN_SEC \times head\ time_factor$)
 - 체결:** 최저점에서 정지 유지 ($FASTEN_SEC \times head\ time_factor$). 하강 완료(접촉) 시점에 **반제품이 실제로 있는 스테이션에 대해서만** 측정 API를 호출하고, 응답 수신 후 해당 반제품에 나사 표시를 추가한다. 빈 슬롯은 공타 — 아무 데이터도 발생하지 않는다
 - 상승:** 두 헤드가 동시에 올라간다 ($HEAD_UP_SEC \times head\ time_factor$)
 - 이송:** 헤드가 상단에서 대기하는 동안 컨베이어가 라인 위 모든 반제품을 한 슬롯씩 오른쪽으로 전진시킨다 ($INDEX_SEC \times conveyor\ factor$). 이 단계에서 세 가지가 동시에 일어난다:
① 투입 대기 수량이 남아 있으면 새 반제품이 투입구 개구부에서 미끄러져 나와 IN 슬롯에 자리 잡는다 ② ST2에 있던 반제품은 OUT 방향으로 이동해 대차 안으로 미끄러져 들어가며 사라진다 ③ 대차에 들어간 반제품은 완성 카운터 +1 및 `/api/unit-complete` 호출로 반영된다
- 종료: 목표 수량이 모두 대차에 도착하면 상태 머신 정지. 마지막 반제품이 라인 위에 있는 동안에도 두 헤드는 끝까지 공타로 함께 움직인다
 - 구현 지침:** 두 헤드가 별도의 타이머·애니메이션 루프를 갖지 않도록, 단일 상태 머신(하나의 타이머 또는 `requestAnimationFrame` 루프)이 라인 전체 상태를 소유하고 두 헤드는 같은 상태 변수를 읽어 렌더링만 한다. 미세한 위상 어긋남도 허용하지 않는다

F3. 측정 및 판정 (백엔드) — 생산 조건 반영

- `POST /api/measure` — body: `{lot_id, unit_sn, station}`. 조건은 Lot에 저장되어 있으므로 요청에 다시 넣지 않는다
- 측정값 생성 공식:**

```
temp_bias = +TEMP_BIAS (온도 > 적정 상한) | -TEMP_BIAS (온도 < 적정 하한) | 0
(적정 범위 내)
value = round( gauss(mean, std × head_speed의 std_factor) + temp_bias, 2 )
```

- 온도 이상 → 분포 전체가 한쪽으로 치우침 (한쪽 규격 위반 증가)
- 헤드 속도 빠름 → 산포 증가 (양쪽 규격 위반 모두 증가)
- 레일 속도 → 측정값에 영향 없음 (생산 속도만 변화)
- $LSL \leq \text{값} \leq USL$ 이면 OK, 아니면 NG 판정
- 판정 결과를 즉시 엑셀에 기록(F4)하고 응답으로 반환: `{value, judgment}`

F4. 엑셀 기록 (openpyxl)

- Lot 시작 시 `output/LOT-YYYYMMDD-NNN.xlsx` 생성
- 시트 1 "측정기록" — 컬럼: 순번 | 반제품SN | 공정 | 측정항목 | 측정값 | LSL | USL | 판정 | 기록시각
 - 측정 이벤트마다 1행 추가 (반제품당 2행 발생)
 - NG 행의 측정값 셀: 배경 빨간색(PatternFill, FFC7CE) + 판정 셀에 "NG"
 - 조건은 Lot당 고정이므로 측정 행마다 반복 기록하지 않는다
- 시트 2 "NG추적" — 컬럼: 반제품SN | NG공정 | 측정값 | 발생시각. NG 발생 시에만 추가
- 시트 3 "Lot요약" — Lot 완료 시 작성: 생산 조건(온도/헤드 속도/레일 속도), 박스 수, 생산수량, OK수, NG수, 불량률(%), 공정별 통계(평균/표준편차/최소/최대/NG수)
- 매 기록마다 `wb.save()` 호출. 저장 실패(파일 잠금 등) 시 서버가 죽지 않고 재시도 큐에 넣을 것 — 다음 기록 때 함께 저장
- 헤더는 굵게 + 회색 배경, 전체 폰트 Arial

F5. 실시간 현황 패널 (웹 화면 우측)

- 현재 Lot 생산 조건 표시 (온도 · 헤드 속도 · 레일 속도)
- Lot ID, 진행률(완성 n / 목표 N), OK 수, NG 수, 실시간 불량률
- NG 발생 목록 (SN, 공정, 측정값) — 최근 발생이 위로
- 0.5초 폴링(`GET /api/status/{lot_id}`)으로 갱신

F6. Lot 완료 및 보고서 자동 생성

- 목표 수량이 모두 대차 도착 → 백엔드가 자동으로 보고서 생성 시작
- 1단계 (파이썬): 통계 집계 — 생산 조건, 박스 수, 생산수, OK/NG 수, 불량률, 공정별 평균/표준편차/최소/최대/NG 수, NG 반제품 SN 목록
 - 집계 완료 시 조건+집계 결과를 `lots_index.json`에 추가 저장 (서버 재시작 후에도 비교 기능이 동작해야 함)
- 2단계 (LLM): 집계 결과 JSON(조건 포함)만 프롬프트에 담아 사내 LLM에 전달, 3~5문장의 해석 코멘트 생성

- 원시 측정 데이터 전체를 LLM에 넣지 말 것. 집계값만 전달.
- 프롬프트에 명시할 것: "숫자를 새로 만들거나 계산하지 말고, 제공된 수치를 근거로 해석과 권고만 작성하라. 생산 조건(온도·속도)과 결과의 연관 가능성을 언급하라"
- LLM 호출 실패 시: 보고서에 "AI 코멘트 생성 실패" 문구를 넣고 보고서 생성 자체는 계속 진행 (LLM 장애가 전체를 막으면 안 됨)
- 3단계 (출력): HTML 보고서 페이지 생성
 - 구성: Lot 정보 + 생산 조건 카드 → 요약 수치(카드형) → 공정별 통계 표 → NG 목록 표 → AI 해석 코멘트 박스 → 엑셀 파일 다운로드 링크
 - 모든 수치는 파이썬 집계값에서 직접 렌더링. LLM 출력에서 숫자를 가져오지 말 것
 - `output/L0T-xxx_report.html`로 저장 + 화면에서 [보고서 보기] 버튼으로 열람
- 보고서 확인 후 [새 Lot 시작] 버튼으로 초기화

F7. Lot 비교 기능 (신규)

- 화면에 [Lot 비교] 메뉴/버튼 → 완료된 Lot 목록(`GET /api/lots`, `lots_index.json` 기반)을 조건·불량률과 함께 표시
- 사용자가 Lot 2개를 선택 → `GET /api/compare?lot_a=...&lot_b=...` → 비교 보고서 생성
- 비교 보고서 구성 (3층 구조):
 1. 수치 표 (파이썬): 두 Lot의 조건과 결과를 나란히 배치 — 온도 | 헤드 속도 | 레일 속도 | 생산수 | NG수 | 불량률 | 공정별 평균 | 공정별 표준편차. 불량률이 낮은 쪽 셀을 강조 표시
 2. 규칙 기반 결론 문장 (파이썬, LLM 미사용): 템플릿 예 — "Lot A(24°C, 헤드 보통)가 Lot B(29°C, 헤드 빠름)보다 불량률이 X.X%p 낮습니다." 조건이 동일한 항목과 다른 항목을 명시
 3. LLM 해석: 두 Lot의 집계 JSON을 전달해 "어떤 조건 차이가 결과 차이에 기여했을 가능성이 있는지" 해석 코멘트 생성. 숫자 생성 금지 규칙 동일. 단일 비교는 엄밀한 인과 증명이 아니므로 단정 대신 가능성 서술을 요구할 것
- `output/COMPARE-{lotA}-{lotB}.html`로 저장 + 화면에서 열람
- 조건이 완전히 동일한 두 Lot을 비교하는 경우도 허용 (반복 재현성 확인 용도)

6. API 설계

Method	Path	설명
POST	/api/lot	Lot 생성. body: {boxes, temperature, head_speed, conveyor_speed} → {lot_id, quantity, tact_times}
POST	/api/measure	측정 실행. body: {lot_id, unit_sn, station} → {value, judgment}
POST	/api/unit-complete	반제품 완성(대차 진입) 통지. body: {lot_id, unit_sn} — 마지막 반제품이면 보고서 생성 트리거
GET	/api/status/{lot_id}	현황 조회 (F5 폴링용, 생산 조건 포함)

Method	Path	설명
GET	/api/report/{lot_id}	Lot 보고서 HTML 반환
GET	/api/lots	완료된 Lot 목록 (조건+불량률 요약)
GET	/api/compare? lot_a=&lot_b=	비교 보고서 생성.반환
GET	/	메인 페이지 (정적 파일)

- tact_times: 백엔드가 조건 배율을 계산해 프론트에 내려주는 실제 택트 시간들. 배율 계산 로직은 백엔드 한 곳(config 기반)에만 존재해야 하며 프론트에 중복 구현하지 말 것

7. 비기능 요구사항

- 모든 설정값은 config.py 한 곳에서만 수정하도록 집중
- 예외 발생 시 서버 프로세스가 죽지 않을 것 (측정 API는 어떤 경우에도 200 또는 명확한 에러 JSON 반환)
- 프로젝트 구조: (main.py)(FastAPI), (config.py), (excel_writer.py), (report.py), (compare.py), (llm_client.py), (static/)(index.html, app.js, style.css)
- 외부 의존성 최소화: fastapi, uvicorn, openpyxl, openai, python-dotenv 만 사용

8. 개발 단계 (이 순서를 반드시 지킬 것)

Phase 1 — 로직 파이프라인 (화면 없음)

CLI 스크립트 하나로: 박스 수 + 생산 조건 입력 → 수량 환산 → SN 생성 → 조건 반영 공식으로 두 공정 측정값 생성.판정 → 엑셀 기록(빨간 셀 포함) → 집계 + lots_index.json 저장 → LLM 코멘트 → HTML 보고서 저장까지 전부 실행. 완료 기준: (python pipeline_test.py 2 --temp 24 --head 보통 --conveyor 보통) 실행 시 측정기록 40행짜리 엑셀과 보고서 HTML 생성. 이어서 (--temp 29) 로 재실행 시 측정값 평균이 눈에 띄게 위로 치우치고 NG가 증가함을 출력으로 확인.

Phase 2 — FastAPI 백엔드화

Phase 1 로직을 6장의 API로 분리 (비교 API 포함). 완료 기준: 테스트 스크립트로 조건이 다른 Lot 2개를 API로 생산 → /api/compare 호출 시 수치 표와 규칙 기반 결론 문장이 포함된 비교 보고서 HTML 생성.

Phase 3 — 프론트엔드 + 애니메이션

조건 설정 패널, SVG 조립라인(기계·반제품·투입구·대차 묘사 규칙 준수), 조건 배율이 반영된 택트 상태 머신, 체결 접촉 시점 ↔ 측정 API 동기화, 나사 표시, 현황 패널 폴링. 완료 기준: 브라우저에서 조건 설정 후 1박스 입력 → 반제품이 투입구에서 나와 라인을 흐르고, 두 헤드가 시종일관 동일한 높이로 함께 움직이며(공타 포함) 애니메이션 완주 → 헤드 속도 "빠름" Lot에서 헤드 왕복이 눈에 띄게 빨라짐 → 완제품이 대차로 들어가며 사라지고 완성 카운터가 정확히 올라감 → 현황 패널 수치가 엑셀 기록과 일치 → 보고서 열람 가능.

Phase 4 — Lot 비교 UI

완료 Lot 목록 화면, 2개 선택 UI, 비교 보고서 열람. **완료 기준:** 24°C/보통 Lot과 29°C/빠름 Lot을 화면에서 선택·비교 → 수치 표(불량률 낮은 쪽 강조) + 규칙 문장 + LLM 해석이 모두 포함된 비교 보고서가 열림.

Phase 5 — 폴리싱 (선택)

금속 광택·기계 아이콘 디테일, 현황 패널·보고서 스타일. **기능·타이밍·공식 변경 금지, 시각 개선만.**

9. 최종 인수 기준

1. 2박스(20개) 입력 시 애니메이션이 끝까지 진행되고 완성 카운터가 20이 된다
2. 엑셀 파일에 측정기록 40행(20개 × 2공정)이 존재하고, NG 행의 측정값 셀만 빨간색이다
3. NG추적 시트의 행 수 = 현황 패널의 NG 발생 건수 = 보고서의 NG 목록 수가 모두 일치한다
4. 보고서의 모든 수치가 엑셀 Lot요약 시트와 일치하며, AI 코멘트에 등장하는 수치가 집계값과 모순되지 않는다
5. LLM 서버를 끈 상태에서도 보고서가 (코멘트 실패 문구와 함께) 정상 생성된다
6. 엑셀 파일을 열어둔 채 생산을 진행해도 서버가 죽지 않는다
7. **동기화 검증:** 생산 전 구간에서 두 헤드의 수직 위치가 항상 동일하다 (시작 직후·종료 직전의 공타 구간 포함). 두 헤드가 서로 다른 타이머로 구현되어 있으면 실패로 간주한다
8. 대차로 들어가는 모든 반제품에는 그 직전까지 나사 표시가 정확히 2개(왼쪽·오른쪽) 붙어 있다
9. 반제품은 투입구 개구부에서 나와 등장하고, 완제품은 대차 안으로 들어가며 사라진다. 대차에 쌓인 완제품이 화면에 그려져서는 안 되며, 완성 수량은 대차 옆 카운터 숫자("완성 n/N")로만 표시된다
10. **조건 영향 검증:** 5박스(50개) 기준 — 29°C Lot의 공정별 평균이 24°C Lot 대비 약 +0.5 부근으로 치우친다. 헤드 "빠름" Lot의 공정별 표준편차가 "보통" Lot보다 큰 경향을 보인다 (난수이므로 경향 확인, 정확한 값 요구 아님)
11. 생산 중에는 조건 패널이 잠기며, Lot요약 시트·보고서·비교 보고서 모두에 해당 Lot의 조건이 정확히 표기된다
12. 비교 보고서의 모든 수치는 각 Lot의 Lot요약과 일치하고, 규칙 기반 결론 문장의 불량률 차이(%p)가 수치 표와 정확히 맞는다